

RTL Design Sherpa

pumice on the Nexys A7 The FPGA System: Harness, Builds, Flow -- Version 1.0

September 2026

Table of Contents

1 Document Information....8
1.1 Related documents....8
1.2 Terminology....9
1.3 Revision History....9
1.4 How to read this book....9
2 The System....11
2.1 What is on the bench....11
2.1.1 Figure 1.1: The bench....11
2.2 The board and the part....11
2.3 The single USB connector....12
2.4 The clock chain, and why it fixes every later number....13
2.5 Where the DUT ends and the harness begins....13
3 The Harness....14
3.1 The shape of it....14
3.1.1 Figure 2.1: The harness block map....14
3.2 The host front-end....15
3.3 harness_csr: the registers that define a run....16
3.4 The engine spine, and why it is a separate block....18
3.5 The DFI tail....19
3.6 Observability, and one honest compromise....19
3.7 Why the APB CDC is an async FIFO and not a handshake....20
4 One Harness, Five Targets....21

4.1 The short answer....21
4.1.1 Figure 3.1: One harness source, five targets....21
4.2 The five targets....22
4.3 What the ILA builds change, and what they do not....23
4.4 The LiteDRAM build is the point of the whole structure....23
4.5 The simulation twin....24
4.6 The frequency define is part of the build identity....24
4.7 What is parameterized, and the one that bites....24
5 Host Tools and Flow....26
5.1 The shared transport....26
5.2 From a command to a number....27
5.2.1 Figure 4.1: From host command to a row of the CSV....27
5.3 Step 1: bring-up, once per bitstream....27
5.4 Steps 2 to 5: configure, run, measure, read back....28
5.5 Step 6: recording, and the arithmetic that has already gone wrong once....28
5.6 Integrity is what makes throughput mean anything....28
5.7 One program, two targets....29
5.8 The bench programs worth knowing....29

List of Figures

Figure 1.1: The bench..... 11
Figure 2.1: The harness block map..... 14
Figure 3.1: One harness source, five targets..... 21
Figure 4.1: From host command to a row of the CSV..... 27

List of Tables

Table 1: Table 0.1: Document information....8
Table 2: Table 0.2: Related documents....8
Table 3: Table 0.3: Revision history....9
Table 4: Table 1.1: Board and memory device....11
Table 5: Table 2.1: The bridge address map....15
Table 6: Table 2.2: harness_csr, the registers that define a run....16
Table 7: Table 2.3: The engine spine....18
Table 8: Table 3.1: The five targets built from one harness source....22
Table 9: Table 3.2: Build-time and run-time parameters....24
Table 10: Table 4.1: The shared host transport....26
Table 11: Table 4.2: The cached leveling result for this board....27
Table 12: Table 4.3: The host programs....29

List of Waveforms

No waveforms in this document.

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

1 Document Information

Table 0.1: Document information

Field	Value
Title	pumice on the Nexys A7 – The FPGA System
Version	1.0
Date	2026-09-29
Scope	The DDR2 characterization system: board, the harness around the controller, the five things built from one harness source, and the host flow
Not in scope	The pumice controller itself (HAS/MAS under `projects/components/mem-ctrl-ip/research-ip/pumice-ddr2-lpddr2/docs/`); the measured results (`build-perf/results/`, `ddr2-characterization/char_results/`)
Status	Current; written against the tree after the `mem-ctrl-ip` and `fabric-gen-ip` family moves

1.1 Related documents

Table 0.2: Related documents

Document	Where	What it gives you
pumice HAS / MAS	`projects/components/mem-ctrl-ip/research-ip/pumice-ddr2-lpddr2/docs/`	the controller's own architecture; this book treats it as the DUT
Harness address map	`build-perf/host/ADDRESS_MAP.md`	the authoritative register decode, mirrored from the RTL
LiteDRAM flow README	`ddr2-characterization/flows-litedram-uart/README.md`	the A/B reference build and its config

Document	Where	What it gives you
DDR2 char guide	`ddr2-characterization/docs/ddr2_char_guide/`	the older operator guide (v0.90)
STREAM / RAPIDS system books	`projects/fpga-systems/Genesys2/*/docs/*_fpga_system/`	the same book for the Genesys 2 boards

1.2 Terminology

DFI DDR PHY Interface. The boundary between the controller and the PHY, and the same boundary the DV repo's BFM drives in simulation.

a7ddrphy LiteDRAM's Artix-7 DDR PHY, reused verbatim. It owns the IOB serdes, bitslip and IDELAY training that are FPGA-specific and deliberately out of the controller's scope.

Harness Everything in the bitstream that is not the DUT: the UART bridge, the address bridge, the CSR block, the stimulus generators, the meters and the displays.

Engine spine `char_engine_block.sv` – the DUT-agnostic part of the harness: generator registers, generator array, AXI merge, bus meters and latency histograms.

Design point 75 MHz system clock (`PUMICE_SYS_75`), 1:2 gear to a 150 MHz DDR2 clock, 300 MT/s, BL4 on a 16-bit bus: 600 MB/s theoretical peak.

1.3 Revision History

Table 0.3: Revision history

Version	Date	Change
1.0	2026-09-29	First edition. Written after the family-directory moves, so every path in it is `mem-ctrl-ip/` and `fabric-gen-ip/`.

1.4 How to read this book

Chapter 1 is the bench: one USB connector, one DRAM part, and the clock chain that fixes the peak every later number is measured against. Chapter 2 is the bitstream around the DUT, block by block, with the address map every host tool depends on – and, for each block, why it is shaped the way it is. Chapter 3 is the chapter that answers the question people actually ask: the harness RTL does *not* change per build, and the chapter says what does. Chapter 4 is the measurement loop, including two traps that have each cost a wrong conclusion.

Every diagram is a Graphviz source under `assets/graphviz/`, rendered to PNG by `regenerate_all_graphviz.sh`. Edit the `.dot`, re-run the script, never hand-edit a PNG.

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

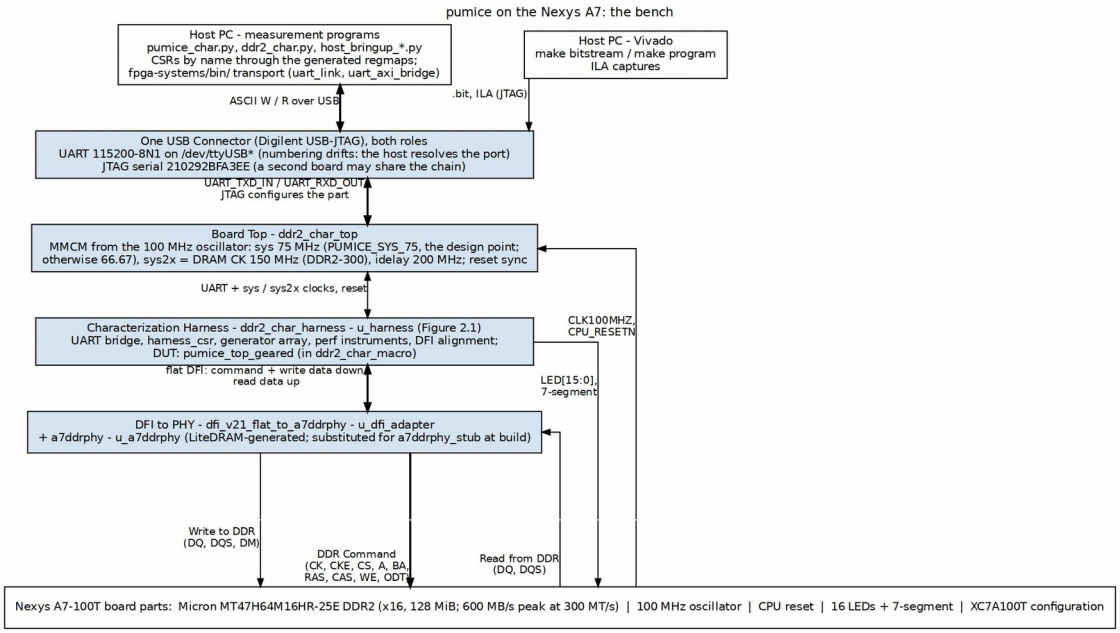
2 The System

2.1 What is on the bench

One Nexys A7-100T, one USB cable, one DDR2 part soldered to the board, and a host program on a Linux workstation. The device under test is the pumice DDR2/LPDDR2 memory controller; everything else in the bitstream exists to put traffic through it and count what comes back.

The split matters and this book keeps to it. The controller's architecture – its AXI front end, its scheduler, its CAMs, its DFI layer – is the subject of the pumice HAS and MAS. What follows is the *apparatus*: the part you would have to rebuild if you wanted to measure some other controller, which is exactly what the LiteDRAM comparison build does.

2.1.1 Figure 1.1: The bench



The bench: host, one USB connector, the Artix-7 and the DDR2 part

Source: 01_board_and_host.dot

2.2 The board and the part

Table 1.1: Board and memory device

Item	Value	Source
Board	Digilent Nexys A7-100T	`projects/fpga-systems/boards/nexys_a7_100t/`

Item	Value	Source
		board_info.md
FPGA	Xilinx Artix-7 XC7A100T-1CSG324C	same
Oscillator	100 MHz, single	same
DRAM	Micron MT47H64M16HR-25E, DDR2, x16, single rank, 128 MiB	ddr2-characterization/README.md
DDR2 banks	Bank 34 at 3.3 V LVCMOS33, Bank 35 at 1.5 V SSTL15	board_info.md

Note: board_info.md records the DRAM as "128 Mb", which is the bit count of the wrong generation of part. MT47H64M16 is 64 M addresses by 16 bits = 1 Gbit = 128 MiB, which is what the characterization README states and what the address map assumes. Where the two disagree, the part number wins.

2.3 The single USB connector

One connector does both jobs: it carries the JTAG chain that programs the device and drives the ILA, and it carries the UART the host talks to. The practical consequences are what the tooling has to cope with:

- **JTAG is addressed by serial number**, 210292BFA3EE, because both this board and the Genesys 2 sit on the same chain on this bench.
- **The UART is not addressed by a constant.** It appears as some /dev/ttyUSB*, the numbering drifts between plug-ins, and a second board may be present. The host resolves the port rather than hardcoding it; the cdc_demo driver on the same board carries the same comment for the same reason.

This is the one place where the Nexys A7 is simpler than its sibling. The Genesys 2 needs two chips – an FT2232 for JTAG and a separate FT232R for the UART, each enumerating independently – and its system book has a whole section on telling them apart.

2.4 The clock chain, and why it fixes every later number

The board has one 100 MHz oscillator. From it the design derives a 75 MHz system clock and, through a 1:2 gear, a 150 MHz DDR2 clock giving 300 MT/s on the bus. With BL4 on a 16-bit bus that is:

```
300 MT/s x 2 bytes = 600 MB/s theoretical peak
```

That 600 MB/s is the denominator of every percentage in every findings page. It is not a target, it is arithmetic; quoting a throughput without it is how a number stops meaning anything, which is why the house rule is that a bandwidth table carries the measured figure *and* the peak.

The 75 MHz profile is selected by the `PUMICE_SYS_75` define. It is the design point, and `build-perf/fpga/tcl/create_project.tcl` says so in as many words – it prints the frequency profile at build time and labels the alternative, `PUMICE_SYS_75=0` at 66.67 MHz, as *not* the board design point. A bitstream built without the define will work and will report lower numbers, and a reader comparing those against the design-point results would draw a false conclusion.

Important: the timing margin at 75 MHz is deliberately thin. A positive worst-negative-slack of a few tens of picoseconds is the design point being met, not a near miss – the most recent measurement build closed at WNS +0.031 ns. Report the sign and the value; do not "fix" a thin positive margin.

2.5 Where the DUT ends and the harness begins

The boundary is DFI v2.1, and it was chosen so that the same boundary is exercised in three places: by the DV repo's DFI BFM in simulation, by the `a7ddrphy_stub` in the harness's own cosim, and by LiteDRAM's real `a7ddrphy` on the board. Code that passes in cocotb against the BFM should pass on hardware against the PHY, modulo PHY-side training – and PHY training is precisely what Chapter 4's bring-up sweeps exist to settle.

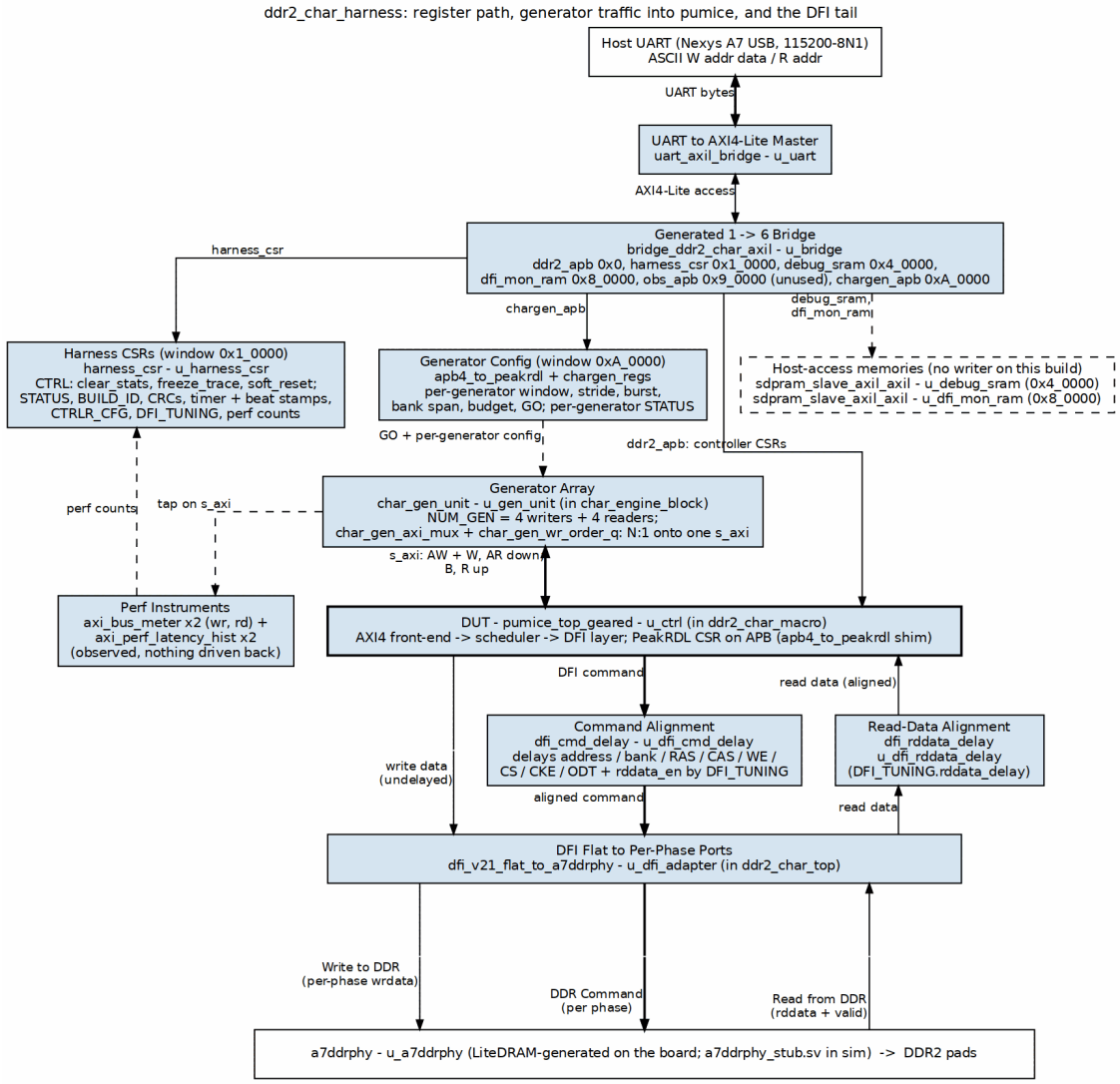
The PHY itself is out of scope for the controller family on purpose. IOB serdes, bitslip, IDELAY tap calibration: all FPGA-specific, none of it something a DDR2 controller should own. So the harness reuses LiteDRAM's `a7ddrphy` verbatim rather than reimplementing it.

3 The Harness

3.1 The shape of it

The harness is a host front-end, a DUT-agnostic engine spine, the DUT, the DFI tail, and two kinds of observability. Read the figure left to right and the whole apparatus is one sentence: UART bytes become register writes, register writes start generators, generators put AXI traffic through the controller, meters count what the controller did with it, and the host reads the counters back.

3.1.1 Figure 2.1: The harness block map



ddr2_char_harness: the blocks around the DUT, and the address map

Source: 02_harness_blocks.dot

3.2 The host front-end

`uart_axil_bridge` turns the UART byte stream into a 32-bit AXIL master. `bridge_ddr2_char_axil` – a *generated* bridge, from `ddr2_char_framework/rtl/bridges/configs/bridge_ddr2_char_axil.toml` – fans that one master out to six slaves at fixed bases.

Table 2.1: The bridge address map

Slave	Base	Size	Protocol	Purpose
`ddr2_apb`	`0x0000_0000`	4 KB	APB	pumice controller CSR (PeakRDL-generated)
`harness_csr`	`0x0001_0000`	4 KB	AXIL	harness control, timer, engine cfg, perf readback
`debug_sram`	`0x0004_0000`	256 KB	AXIL 64b	host-access memory; no writer on this build (the trace ring is unused, see below)
`dfi_mon_ram`	`0x0008_0000`	4 KB	AXIL	placeholder for DFI command observability; no writer on this build, host access only
`obs_apb`	`0x0009_0000`	4 KB	APB	expansion slot for an interface observer; unused on

Slave	Base	Size	Protocol	Purpose
chargen_apb	0x000A_0000	4 KB	APB	this build chargen_regs : per-generator configuration, status and GO

Two details in that map are deliberate rather than accidental. The 256 KB debug_sram window spans 0x4_0000..0x7_FFFF, which is why dfi_mon_ram sits at 0x8_0000 and not at 0x5_0000 – placing it lower would have overlapped. And the slot at 0x2_0000..0x3_FFFF is unallocated: it used to hold a desc_ram, which became unnecessary once the pattern-generator engines covered the workload class the descriptor mode was reserved for.

Important: build-perf/host/ADDRESS_MAP.md mirrors this decode for convenience, and says outright that the RTL is authoritative. If the two disagree, the fix is to the document, not to the hardware.

3.3 harness_csr: the registers that define a run

Table 2.2: harness_csr, the registers that define a run

Off	Register	R/W	What it does
0x00	CTRL	RW	clear_stats, freeze_trace (latch), soft_reset; bits 0/1 (start_wr, start_rd) are retired – launch is chargen_regs.GO
0x04	STATUS	R	per-direction done/error, any_error, init_done, init_fail
0x10/0x14	CRC_EXPECTED / CRC_ACTUAL	R	write-engine expected CRC, read-engine actual CRC

Off	Register	R/W	What it does
0x18	CRC_MATCH	R	exp==act, both valid bits, and beats_mism != 0
0x24	BEATS_MISM	R	mismatched beat count from the read engine
0x20	BUILD_ID	R	0x44445232, the ASCII DDR2 – proves you are talking to this build
0x28-0x5C	timer block	RW/R	64-bit cycle counter, beat-count stop trigger, and first/last R and W beat stamps
0x60	CTRLR_CFG	RW	memtype (DDR2/LPDDR2), t_phy_wrlat, t_rddata_en, rd_in_order
0x64	CTRLR_CAP	RW	scheduler lookahead and synth-mask caps
0x80-0x8C	PHY CSR passthrough	RW/R	indirect access to the a7ddrphy leveling knobs

The first/last beat stamps are what make a throughput number defensible: they bound the window the bytes actually moved in, so the host divides by a measured interval rather than by "however long the host thought the run took".

BUILD_ID exists because the cheapest way to waste an afternoon is to measure a stale bitstream. A host that cannot read DDR2 back is talking to something else.

Note: RESP_DELAY at 0x3C is present and documented as unwired. axi_response_delay.sv is committed under ddr2_char_framework/rtl/ and carried in the filelist but not instantiated, so

that adding programmable memory latency later is a one-line swap-in rather than a new dependency. A register that reads back what you wrote and changes nothing is a trap unless it is labelled, which is why it is labelled here and in the address map.

3.4 The engine spine, and why it is a separate block

`char_engine_block.sv` is the part of the harness that does not know what the DUT is. It holds the generator registers, the generator array, the AXI merge, and the instruments:

Table 2.3: The engine spine

Block	Role	Why it is shaped this way
`chargen_regs` (PeakRDL)	per-generator address window, stride, burst length, bank span, transaction budget	the stimulus is *described*, not compiled in, so a sweep is a host loop and not a rebuild
`char_gen_unit`	`NUM_GEN` = 4 write and 4 read generator blocks (the default; no build overrides it)	each spans `NUM_BANKS / NUM_GEN` banks, so **bank concurrency is a property of the stimulus**, not of the controller – that is what makes the bank/gap knee measurable
`char_gen_axi_mux` + `char_gen_wr_order_q`	N:1 merge onto the DUT's single `s_axi`	see below
`axi_bus_meter`	per cycle: productive, backpressure, starvation, idle	four states that account for every cycle; this is where "the arbiter is idle 33% of the time" came from
`axi_perf_latency_hist`	response-time histograms	a mean hides a bimodal distribution, which is exactly what a page-policy change produces

The merge deserves its own paragraph because it replaced something. It used to be two *generated* 2x1 bridges. They were not removed because their routing was wrong – at 2x1 that routing is a combinational grant-lock round robin and the replacement works the same way. They were

removed because of the four generated adapters wrapped around them, which imposed a `bridge_cam DEPTH(16)` cap on outstanding transactions and two skid stages per direction. On a harness whose measurement *is* latency, the apparatus was setting the answer. The generated configurations are still in the tree so both paths can be built and compared; nothing in this build reads them.

3.5 The DFI tail

Between the controller and the pads sit three things, and all three are harness, not DUT:

- `dfi_cmd_delay` and `dfi_rddata_delay` – programmable command and read-data alignment. These are the knobs the bring-up sweeps turn, and the reason a working board configuration is a *tuple* rather than a single number.
- `dfi_v21_flat_to_a7ddrphy` – flat DFI to the PHY's per-phase ports.
- the PHY itself: `a7ddrphy_stub.sv` for verilator and cocotb, and LiteDRAM's generated `a7ddrphy` on hardware. Vivado excludes the stub and substitutes the real PHY at build time. The generated PHY is never hand-edited – when the PHY DQ tristate had to be observed, the ILA was marked on the *synthesized netlist* instead, which is what the `ila_phy` build is.

The PHY leveling knobs are driven by *firmware*, not by a hardware state machine: there are 13 of them, reached indirectly through the four `PHY_CSR_*` registers, with the knob map in `rtl-vivado/a7ddrphy/a7ddrphy_csr_map.txt`. Only meaningful on hardware; the sim stub ignores the writes.

3.6 Observability, and one honest compromise

`dfi_mon_ram` is reserved for DFI command observability but has no writer on this build – only the host reads and writes it – and `seven_seg_4digit` plus `led_status_driver` give state at the bench with no host attached – a heartbeat and a pass/fail byte are enough to tell "hung" from "running" across the room.

`debug_sram` is the compromise. The full-size 256 KB trace ring would have needed roughly 44 K LUT-as-distributed-RAM cells against the 100T's 19 K sites – 2.4x over the device – and it blocked `place_design` outright. The backing store is therefore shrunk to `DEBUG_SRAM_WORDS=512` (256 x 64-bit = 2 KB) and the trace ring is not used on this build. The 256 KB *address window* is unchanged, so accesses above 2 KB alias back into the ring rather than erroring.

Important: that aliasing is the dangerous part. A reader who downloads the window and finds plausible-looking repeated data has not found a trace; they have found the same 2 KB four hundred times over. Raise `DEBUG_SRAM_WORDS` in `ddr2_char_harness.sv` on a device with headroom before trusting a trace from this window.

3.7 Why the APB CDC is an async FIFO and not a handshake

The controller's CSR window reaches it through `apb4_slave_cdc` and `peakrdl_to_cmdrsp`. The CDC inside `apb4_slave_cdc` is a gray-pointer async FIFO (`gaxi_fifo_async`), deliberately not a toggle handshake, and the reason is a reset asymmetry specific to this harness: the APB side runs on `presetn` and the core side on `aresetn`, and `CTRL.soft_reset` pulses **only the core side**. A toggle-parity handshake desynchronizes across that asymmetric reset and leaves the response stream permanently offset by one transaction – every readback returns the answer to the previous question. An async FIFO with gray pointers re-synchronizes instead of latching a parity error.

This is recorded in the filelist that pulls the CDC in, next to the dependency it justifies, which is the right place for it: the next person to "simplify" that CDC reads the reason before they do it.

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

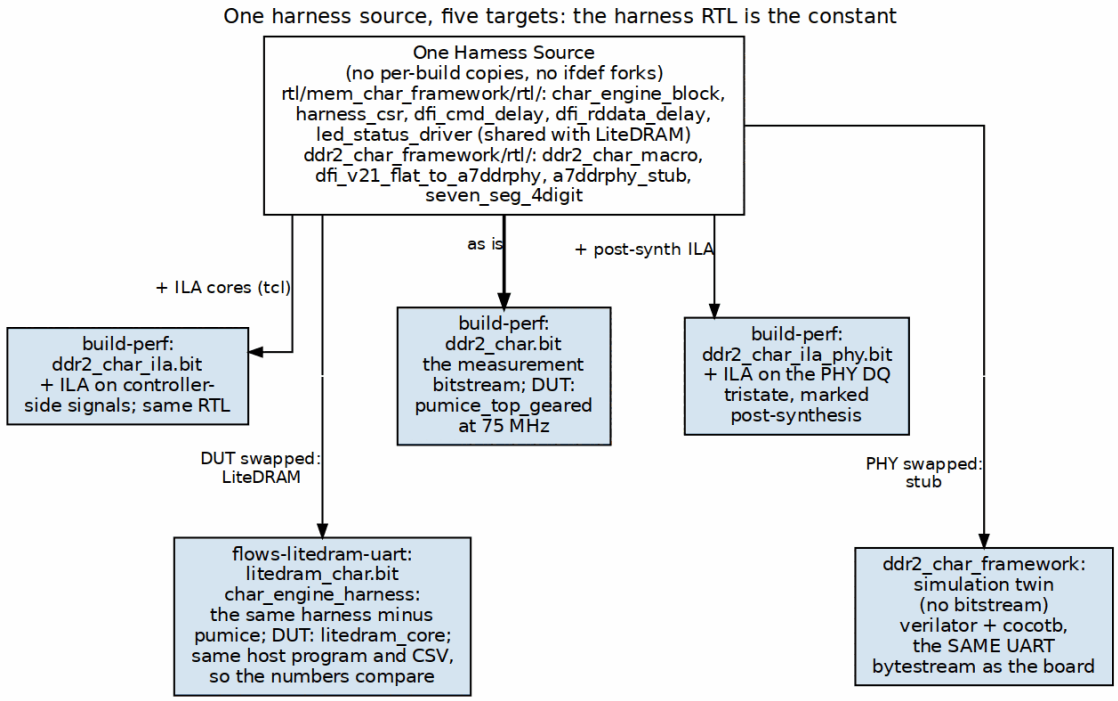
4 One Harness, Five Targets

4.1 The short answer

The harness RTL does not change per build. There is one copy of the engine spine, the CSR block, the DFI delays and the LED driver, in `projects/fpga-systems/rtl/mem_char_framework/rtl/` (shared with the LiteDRAM flow), plus the pumice macro, the DFI-to-PHY adapter, the PHY stub and the 7-segment driver in `ddr2_char_framework/rtl/`, and every target compiles that same source. There are no per-build copies and no `ifdef` forks of the engine.

What changes between targets is one of exactly three things: the debug cores added after synthesis, the DUT behind the AXI port, or the PHY model. Each is worth a section, because each answers a different question.

4.1.1 Figure 3.1: One harness source, five targets



One harness source, five targets. The harness RTL is the constant.

Source: 03_build_variants.dot

4.2 The five targets

Table 3.1: The five targets built from one harness source

Target	Artifact	What differs	What it is for
`build-perf` base	`ddr2_char.bit`	nothing – this is the reference	the measurement build; every published number comes from here
`build-perf` + ILA	`ddr2_char_ila.bit`	ILA cores added by tcl	watching controller-side signals when a number is surprising
`build-perf` + PHY ILA	`ddr2_char_ila_phy.bit`	ILA marked on the *synthesized netlist*	watching the PHY's DQ tristate, which has no RTL you may edit
LiteDRAM A/B	`litedram_char.bit`	the DUT, and the PHY comes with it	is our controller good, compared to what?
simulation twin	none (verilator/cocotb)	the PHY is a stub	validating a host program before it touches hardware

`ddr2-characterization/flows-ours-uart/` was the retired "ours over UART" flow and is **deleted as of 2026-09-30**. It is described here because references to the path outlive the directory, and because what happened to it is a worked example of a real hazard.

Its RTL had already moved to `build-perf/`. What remained was 203 MB of build output, two bitstreams, and 42 CSVs of ILA captures and timing scans from the July 2026 bring-up – **none of it tracked in git**, and 41 of those CSVs existing nowhere else in the tree.

Important, and the reason this paragraph is longer than the directory deserves: an earlier edition of this book called that directory "vestigial: 0 tracked files, only `__pycache__`". The tracked count was right and the rest was wrong – it was written after listing two of its subdirectories and generalising from them. On that description the directory was very nearly deleted as build detritus, and what would have gone with it were the captures behind the three x16 bugs and the read-valid skew fix.

The captures were removed deliberately once they had served their purpose, which is the right ending. The wrong ending was available and close. **“Nothing tracked” is not “nothing there”, and a directory listing of two subdirectories is not an inventory.**

4.3 What the ILA builds change, and what they do not

Neither ILA build edits the harness. `ddr2_char_ila.bit` adds debug cores to controller-side signals through tcl at build time.

`ddr2_char_ila_phy.bit` is the more interesting one, and the reason it exists is a constraint rather than a preference. The signal worth watching for read/write turnaround is the PHY’s DQ output enable – whether the FPGA is driving the DQ wires while the DRAM is returning data. That signal lives inside LiteDRAM’s generated `a7ddrphy_generated.v`, which must not be hand-edited: it is generated output, and editing it plants a change that the next regeneration silently reverts. So the ILA is marked on the **synthesized netlist** instead, matching `*a7ddrphy_dq_oe_delay*` and `*a7ddrphy_dqs_oe_delay*` after synthesis rather than instrumenting the source.

Note: the DFI-boundary signal `w_dfi_rddata_valid` is *not* the same thing. It is a post-capture signal: it tells you a read has been captured, not that the wires were contended. Arming a trigger on it and finding nothing proves nothing about the tristate. The distinction cost a whole ILA campaign before it was understood.

Important: a trigger that never fires is not evidence. When the PHY contention build was brought up, the first three captures came back empty because the trigger armed during the prefill’s write phase. The discipline that came out of it: arm each half of the condition separately first and confirm *those* fire, and only then believe a silent combined trigger.

4.4 The LiteDRAM build is the point of the whole structure

The reason the engine spine is a separate, DUT-agnostic module is this build. `flows-litedram-uart` puts LiteDRAM’s DDR2 controller behind the same AXI port that pumice normally sits on. Its harness, `char_engine_harness.sv`, is described in its own README as build-perf’s harness *minus pumice* – and everything else is the same file:

- the same `uart_axil_bridge`
- the same generated address bridge
- the same `harness_csr`
- the same `char_engine_block` (generator registers, generator array, meters, histograms)
- the same host program and the same CSV schema

Only the controller differs. That is what makes a number from this build and a number from `build-perf` directly comparable, and it is why the comparison is worth anything at all. An A/B where the apparatus also changed measures the apparatus.

The LiteDRAM core is configured to match the pumice design point deliberately: `MT47H64M16` x16, the Xilinx A7DDRPHY, 75 MHz system with 1:2 gearing to DDR2-300 – the same profile `PUMICE_SYS_75` selects – `cmd_buffer_depth` 16, and one 64-bit AXI user port. Matching the configuration is part of matching the experiment.

LiteDRAM brings its own PLL, its own PHY and its own initialization, which is why this build's top (`litedram_char_top.sv`) puts the harness on LiteDRAM's `user_clk` and wires `m_axi` to `user_port_axi_0`. It also vendors a `VexRiscv.v` alongside the core, because LiteDRAM's self-initialization BIOS runs on a soft CPU.

Note: this build also proves the board itself. When pumice could not bring DDR2 up, a LiteDRAM memtest passing on the same board and the same part is what separated "our controller is wrong" from "the pins, the PHY or the DRAM are wrong". A reference build that works is a diagnostic instrument, not just a competitor.

4.5 The simulation twin

`ddr2_char_framework` builds no bitstream. It compiles the same harness with `a7ddrphy_stub.sv` in place of the PHY and `verilator_xilinx_stubs` supplying primitives like `BUFG`, and drives it from cocotb.

Its value is that it speaks **the same UART byte stream as the board**. The host program is not rewritten for simulation; it is pointed at the simulator. A program that works in cosim and fails on hardware has isolated the difference to the PHY or the DRAM, which is a small search space. That equivalence is the reason the framework exists and the reason it is the gate that runs before any controller RTL change is committed.

4.6 The frequency define is part of the build identity

`PUMICE_SYS_75` is not a tuning knob to be left at whatever it happened to be. `create_project.tcl` prints the frequency profile at build time and labels the alternative explicitly as not the design point. Two bitstreams that differ only in that define produce two sets of numbers that must never be compared, and the only defence is that the build says which one it is.

4.7 What is parameterized, and the one that bites

Table 3.2: Build-time and run-time parameters

Parameter	Where	Why it is not a constant
`PUMICE_SYS_75`	build define	75 MHz design point vs

Parameter	Where	Why it is not a constant
		the 66.67 MHz alternative
`DEBUG_SRAM_WORDS`	`ddr2_char_harness.sv`	shrunk to 512 to fit the 100T; raise it on a bigger device
`NUM_GEN` / bank span	`chargen_regs` at runtime	bank concurrency is stimulus, so it must be a host-settable sweep
generator window, stride, burst, budget	`chargen_regs` at runtime	a sweep should be a host loop, never a rebuild

The last two rows are the design decision that makes the system usable: almost everything a characterization run wants to vary is a *runtime* register, so a full sweep is one bitstream and a host loop. Only the things that cannot be runtime – the clock profile and the size of a memory – are build-time.

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

5 Host Tools and Flow

5.1 The shared transport

Nothing about the byte-level transport is specific to this project. It lives in `projects/fpga-systems/bin/` and every board on this bench uses it:

Table 4.1: The shared host transport

Module	Role
`uart_link.py`	frame and checksum the serial link
`uart_axi_bridge.py`	the host end of `uart_axil_bridge`: reads and writes become AXIL transactions
`board.py`, `fpga_board.py`	port discovery, programming, board identity
`sequence.py`	scripted register sequences, replayable

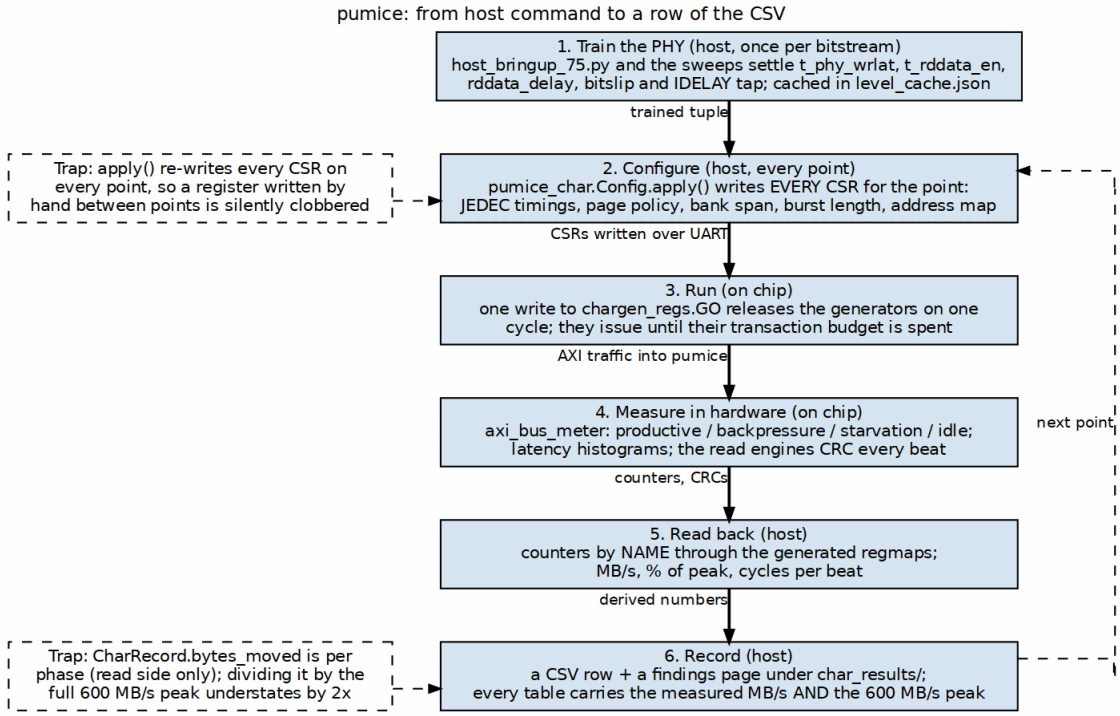
One transport shared across boards is the reason a new project starts at the register map rather than at the serial protocol. It is also why the same `--port auto` discovery logic protects both this project and the CDC demo from the drifting `ttyUSB` numbering described in Chapter 1.

On top of it sit the project-specific layers: `pumice_device.py` and `pumice_master.py` present the controller, and `ddr2_char.py` presents the harness.

Important: these access registers **by name**, resolved through the PeakRDL-generated regmap, never by a hardcoded offset. Offsets churn every time the RDL changes, and a host that hardcodes them fails silently by reading a neighbouring register that happens to decode. `test_harness_regmap_consistency.py` exists to keep the host's view and the generated map from drifting apart.

5.2 From a command to a number

5.2.1 Figure 4.1: From host command to a row of the CSV



From host command to a row of the CSV

Source: 04_measurement_path.dot

The loop is six steps, and the interesting parts are the first and the last.

5.3 Step 1: bring-up, once per bitstream

Before any measurement, the PHY has to be levelled. This is firmware-driven – there is no hardware leveling FSM – so it is a host job, done through the `PHY_CSR_*` passthrough, by `host_bringup_75.py` and the sweep scripts beside it (`host_sweep_rddata_delay.py`, `host_bringup_cmd_delay_sweep.py`, `host_train_per_lane.py`, `host_eye_margin_ab.py`).

The settled result is cached in `level_cache.json`, and the current board records:

Table 4.2: The cached leveling result for this board

Field	Value
`ok`	true
`bitslip`	0

Field	Value
`rd_tap`	8
`rd_window`	0 to 16

A working configuration is a *tuple*, not a single number: bitslip, IDELAY tap, read-data delay and the command/read-enable timings all have to agree. The `rd_window` is the part worth reading – a window of 0 to 16 with the tap at 8 means the chosen tap sits in the middle of a wide passing region, which is what a healthy leveling result looks like. A tap that only works at the edge of a narrow window is a result that will not survive a temperature change.

5.4 Steps 2 to 5: configure, run, measure, read back

`pumice_char.Config.apply()` writes every CSR that defines a measurement point: JEDEC timings, page policy, bank span, burst length, address mapping. Then a single write to `chargen_regs.GO` releases the generators on one cycle, they run until their transaction budgets are spent, and the instruments count in hardware while they do.

Important: `apply()` re-programs **every** CSR on **every** measurement point. A register written by hand between points is therefore silently clobbered on the next one. This is not hypothetical: a tRTW override appeared to take effect and did not, and the only reason it was caught was that the printed configuration line still showed the old value. When overriding something for an experiment, override it where `apply()` will carry it, not after.

5.5 Step 6: recording, and the arithmetic that has already gone wrong once

Results become a CSV row and a findings page under `ddr2-characterization/char_results/` or `build-perf/results/`.

Important: `CharRecord.bytes_moved` is **per phase** – the read side only. Dividing it by the full 600 MB/s peak understates the result by a factor of two. This is not a theoretical hazard: a whole conclusion (“concurrent read plus write is bounded at about 47% of peak by the controller”) was published and then retracted, because the real figure was 95% and 95% is the design point rather than a defect. Every percentage in this system is a chance to make that mistake again, which is why the house rule is that a bandwidth table shows the measured MB/s *and* the theoretical peak, so the division is visible and checkable.

5.6 Integrity is what makes throughput mean anything

The write engine computes an expected CRC and the read engine computes an actual one; `CRC_MATCH` and `BEATS_MISM` are read back with the performance counters. A throughput number from a run that corrupted data is not a throughput number.

Important: a detector that has never fired is not evidence that data is clean. This one had never fired, and proving it worked took deliberately corrupting the analog eye – forcing the read window off its centre – at which point it reported every beat mismatched. Two earlier attempts to inject a fault failed for uninteresting reasons and looked like the detector working. If you add a checker to this harness, make it fail on purpose before you trust it passing.

5.7 One program, two targets

The same host program drives the simulation twin and the board, because the framework speaks the same UART byte stream in both. That equivalence is the system's main defence against wasted board time:

- a host program is debugged in cosim, where a failure is inspectable;
- a difference between cosim and board isolates to the PHY or the DRAM, because everything above DFI is the same source;
- the cosim run is the gate that must pass before controller RTL is committed.

The reverse of that discipline also holds, and it is recorded here because it has bitten: a test run against a stale build directory reports green for the old design. A sub-second "1 passed" is fiction. `make clean-all` before any run that you intend to believe.

5.8 The bench programs worth knowing

Table 4.3: The host programs

Program	What it is for
`pumice_char.py`	the characterization sweeps; produces the CSVs
`ddr2_char.py`	direct harness driver: CSRs, engines, timer, trace
`host_ddr2_smoke.py`	is the board alive and is this the right bitstream
`host_bringup_*.py`	PHY leveling and the timing sweeps behind `level_cache.json`
`host_capture_read.py`, `host_wide_rd_sweep.py`	read-path diagnosis when a number is wrong
`test_pumice_page_stats.py`	checks the page-hit/miss statistics the host derives

`host_ddr2_smoke.py` first, always. It reads `BUILD_ID` and proves the link before anything expensive depends on it.